NATURAL LANGUAGE PROCESSING – CA3

Product Review Sentiment Classification

Field	Details
Programme	BBA (Artificial Intelligence)
Academic Year	2026-27
Team Members	Tanay Chothani - 25030422123
Faculty	Dr. Nabanita Choudhury, Assistant Professor

SYMBIOSIS ARTIFICIAL INTELLIGENCE INSTITUTE (SAII)

Constituent of Symbiosis International (Deemed University)

Lavale, Pune - 412115

1. Abstract

Online clothing shops live and die by their reviews, and nobody can read tens of thousands of them by hand. A sentiment classifier is the obvious answer, but a single accuracy figure can hide a lot. This project builds a positive/negative classifier for product reviews and then asks the more uncomfortable question: does the model treat every product category and every kind of review equally well?

I used the public *Women's E-Commerce Clothing Reviews* dataset (23,486 reviews, released by Nick Brooks). After dropping reviews without text and the ambiguous 3-star reviews, 19,818 reviews remained, of which 88.0% were positive. Text was cleaned and converted into TF-IDF vectors of unigrams and bigrams (14,944 features) and classified with a class-balanced Logistic Regression. Two variants were compared on the same stratified 80/20 split: a plain baseline and a *contrast-aware* variant that gives extra weight to the clause after words such as "but" and "however".

Both variants reached 91.5% accuracy, an F1 of 0.95 on the positive class and a ROC AUC of 0.961. Overall accuracy, however, was the least interesting number. Recall on genuinely negative reviews was 85.2% for the baseline and 85.7% for the contrast-aware model, but it ranged from 54% for Jeans to 97% for Sweaters. Review length made little difference. The contrast-aware idea helped only marginally (2 more complaints caught out of 474). The main outcome of the project is therefore a working, explainable model together with an honest picture of where it can and cannot be trusted.

2. Introduction

2.1 Background and context

The customer reviews are not an option anymore but a natural component of the modern shopping experience. If a person cannot touch the fabric or try the product on, the first thing he will probably do is check the comments of others who already tried that stuff. It works the other way for the sellers as well because they get most of their information about things like sizing or quality from the reviews. However, if you own a medium-sized fashion retail store, then there will be thousands of reviews every month, which no single team can handle. This task is called sentiment analysis. It is one of the classical problems of the natural language processing.

2.2 Motivation for choosing this problem

To hit 90% accuracy in the review dataset – this will not be hard, given the fact that 9 out of 10 reviews have positive sentiment and all positive reviews use the same set of happy words. That is how my interest was raised. While getting a good model is relatively easy when the model is biased towards the majority, true accuracy of such models is in ability to detect negative reviews. As the owner of the business cares about his customers feedbacks, this would be the key factor of building my own model, in which case the main goal will be to analyze the accuracy of the model in different aspects: product category, length of the review and the choice of words.

And there is another peculiarity that caught my eye. Almost all fashion reviews use phrases like "loved the style but after one wash it became ragged". The first part of the phrase is clearly positive while the true sentiment is revealed in the second part. And that is where my interest lies – trying to build an obvious solution to this problem.

2.3 Role of NLP in the proposed solution

This is all about text classification. Reviews are noisy – they have typos, abbreviations, mixed emotions and so forth. First, that noise is converted into a numeric representation via TF-IDF, after which I use a linear classifier to find out what words (or combination thereof) define the gap between positive and negative clients. Why did I opt for a linear classifier, rather than a neural network? Because the latter would be too deep for my liking – each and every prediction would boil down to simple word weights, readable by everyone.

3. Problem Statement and Objectives

Problem statement. Given the text of a clothing review, classify it as positive or negative, and make sure that the classifier is reliable across product categories and types of reviews rather than on average.

The specific objectives were:

1. **Build** a working sentiment classifier on the Women's E-Commerce Clothing Reviews dataset using TF-IDF features and Logistic Regression.
2. **Compare** a baseline pipeline with a contrast-aware pipeline under identical settings and an identical train/test split, so that any difference can be credited to the one change.
3. **Audit** the model by product category and by review length, using recall on negative reviews as the key measure, because the classes are imbalanced.

4. **Explain** the predictions by inspecting the strongest word weights and by breaking down individual predictions.
5. **Document** the limits and a sensible intended use of the model, in the spirit of a model card.

4. Dataset Description

The data is the **Women's E-Commerce Clothing Reviews** dataset, published on Kaggle by Nick Brooks. It contains real, anonymised reviews written by customers of a women's clothing retailer. The copy used here is the file Reviews.csv included in the project repository; the notebook downloads the same data from a public GitHub mirror.

Table 1: Dataset summary

Item	Value
Source	Kaggle: Women's E-Commerce Clothing Reviews (Nick Brooks)
Raw records	23,486 reviews
Reviews used after cleaning	19,818
Positive / negative	17,448 (88.0%) / 2,370 (12.0%)
Median review length	58 words
Train / test split	15,854 / 3,964 (stratified 80/20)
Test set composition	474 negative, 3,490 positive
Language	English only
Link	https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

The file has eleven columns, including the reviewer's age, a title, the review text, a 1-5 star rating, a recommended flag, a count of positive feedback votes and three levels of product hierarchy (division, department and class). This project uses only three of them: **Review Text** as the input, **Rating** to derive the label, and **Class Name** (Dresses, Jeans, Knits and so on) to slice the results during the audit.

Label rule. There is no sentiment column in the data, so the label comes from the star rating: 4 or 5 stars is *positive*, 1 or 2 stars is *negative*, and 3-star reviews are removed because a middle rating is ambiguous. This is a reasonable proxy, but it is a proxy, and Section 11 shows a few places where it breaks down.

Class distribution. Of the raw reviews, 13,131 gave five stars and only 842 gave one star, so the data is heavily skewed positive. After cleaning, the split is 88% positive and 12% negative. This imbalance is the single most important property of the dataset: a model that predicted "positive" for everything would score 88% accuracy while finding none of the complaints. The reviews are also unevenly spread across product classes; Dresses (5,315), Knits (4,021) and Blouses (2,599) dominate, while Intimates, Layering and Trend have well under 150 each. Classes with fewer than 50 reviews were merged into an "Other" group (15 reviews).

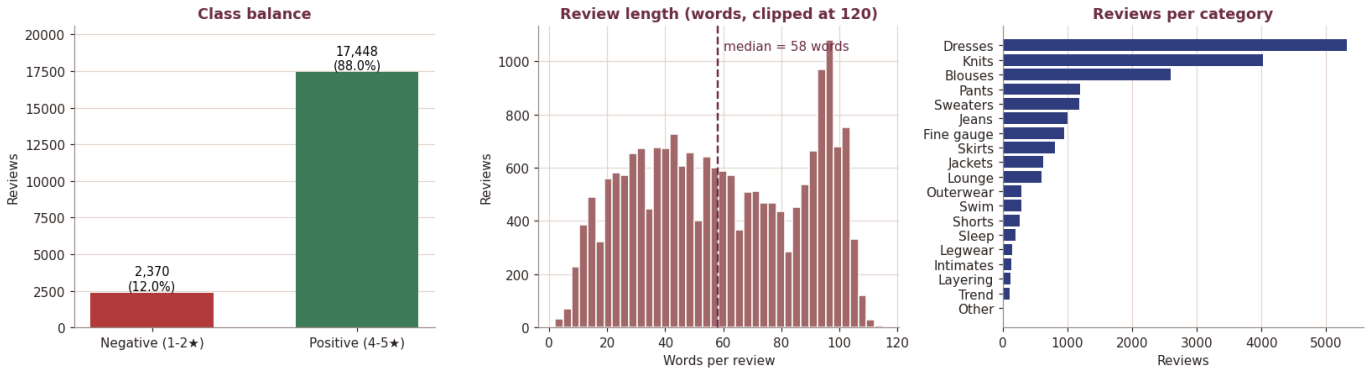


Figure 1: Class balance, review length distribution and reviews per product category (after cleaning)

5. Data Preprocessing

The preprocessing was kept to a minimum, as customer reviews are short and informal texts, and any aggressive normalisation would strip the sentiment-related clues from the text. Table 2 below contains a list of applied preprocessing steps, alongside the reasons for applying them.

Table 2: Preprocessing steps and the reason for each

Step	What was done and why
Remove missing text	Reviews with an empty text field were dropped (845), since there is nothing to classify.
Remove 3-star reviews	A 3-star review is neither clearly happy nor unhappy, and keeping it would add label noise to a binary task.
Create sentiment label	Rating 4-5 became 1 (positive) and 1-2 became 0 (negative).
Merge rare categories	Classes with fewer than 50 reviews were grouped as "Other" so that the category audit does not report on a handful of reviews.
Length group	Each review was marked short or detailed using the median word count (58) as the cut-off, for the length audit.
Lowercasing	"Love" and "love" should count as the same word.
Remove punctuation and digits	Everything outside the letters a-z was replaced with a space. This also removes apostrophes, so "didn't" becomes "didn t"; see Section 11 for what this costs. Digits (sizes, measurements) were removed because they rarely carry sentiment.
Collapse whitespace	Extra spaces left by the previous steps were merged into one.
Stop-word removal	Done inside the vectoriser using scikit-learn's English stop-word list (318 words), to drop very common words that carry no sentiment.

What was not done. I did not perform stemming or lemmatisation. Bigrams in the feature set mean that retaining the original wordforms preserves useful phrases such as wanted love (wanted to love), which proved to be one of the strongest negative signals. There was no special treatment of URL's, hashtags, mentions or emojis since reviews contain almost none, and the data is in English only, so no multilingual or code-mixed handling was necessary.

A detail worth knowing. Looking back at the stop-word list, I checked it and found that it includes words such as not, no, never, but, however, although, yet and very. That is, the vectoriser discards negation and contrast words before the model has seen them. This does not damage the model, since the surrounding content words carry most of the signal, but explains several of the failures discussed in Section 11.

The contrast-aware transform. For the second variant only, an extra stage was added after cleaning. The code finds the first contrast marker (but, however, although, though, unfortunately, except, yet) within the raw review and appends a second copy of everything after it if the trailing clause contains at least two words. This serves to make the part of the review containing the verdict worth approximately double what it would otherwise be. An example is 'I loved the look, but after one wash it shrank terribly' becoming 'i loved the look but after one wash it shrank terribly after one wash it shrank terribly'. About 58% of all reviews, and 71% of negative reviews, contain at least one of these seven words, so the transform touches a large fraction of the data.

6. Feature Representation / Embeddings

The text data had to be converted into numerical representations suitable for use in any model. I applied Term Frequency-Inverse Document Frequency (TF-IDF) algorithm which quantifies each word by its frequency in a particular review down-weighted by its frequency across all reviews. The more common words such as ‘dress’ appear in the corpus, the lower their TF-IDF score is while the rare and informative words such as ‘unflattering’ get a higher weightage.

There were three main reasons for choosing this method. Firstly, TF-IDF is a good baseline model for review sentiment analysis tasks. It is intuitive and has been widely used and tested. Moreover, it creates an interpretable model where each word or combination of words has a particular feature weight that can be easily audited by humans. This would not be the case with many other algorithms, especially Word2Vec and BERT, which have a much higher dimensional feature space. Secondly, it is computationally efficient and trains on 16,000 instances in seconds without requiring GPU resources.

Table 3: TF-IDF vectoriser settings (identical for both variants)

Setting	Value	Reason
ngram_range	(1, 2)	Unigrams plus bigrams capture short phrases such as "too small" or "wanted love".
stop_words	English (318)	Removes very frequent function words.
min_df	5	Ignores terms in fewer than 5 reviews; they are mostly typos and cannot generalise.
max_df	0.9	Ignores terms in over 90% of reviews.

Setting	Value	Reason
max_features	20,000	Upper limit on vocabulary size.
Features kept	14,944	Actual vocabulary after the filters above (fitted on the training set only).

The vectoriser is fitted on the training set only and then applied to the test set, so no information from the test reviews leaks into the vocabulary or the IDF weights.

7. Proposed Methodology / System Architecture

Figure 2 shows the complete workflow. Raw reviews are filtered and labelled, split once into training and test sets, and then passed through one of two text transforms. Both branches share the same TF-IDF vectoriser settings and the same classifier settings, so the only thing that differs between them is the text transform. The trained models are then evaluated overall and on slices of the test data, and their predictions are explained through the learned word weights.

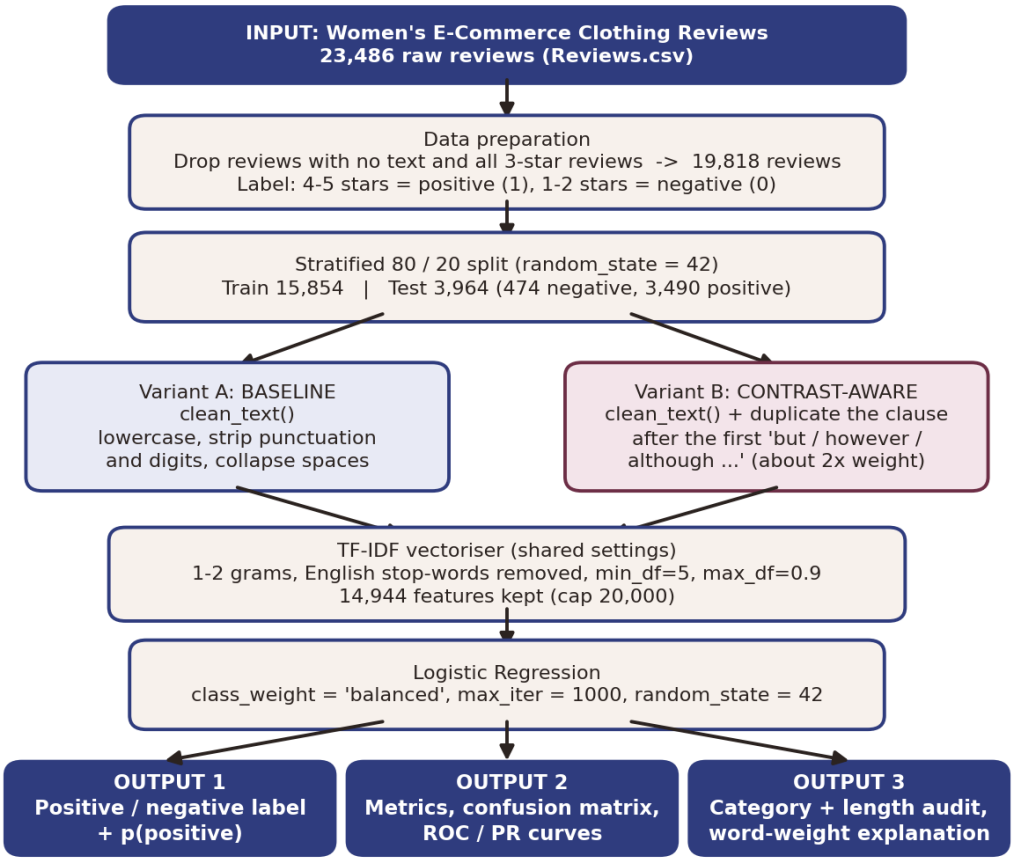


Figure 2: End-to-end workflow of the sentiment classification and audit pipeline

The work was organised in four stages:

- **Prepare:** clean the data, derive labels, category groups and length groups, and make one stratified split.
- **Build:** train the baseline and contrast-aware pipelines on the training set.
- **Audit:** measure accuracy, precision, recall, F1, ROC AUC and average precision overall; then measure negative-review recall by category and by review length.
- **Explain:** read the strongest positive and negative word weights, and decompose individual predictions into per-word contributions.

8. NLP Model / Technique Used

8.1 Logistic Regression

The classifier is scikit-learn's Logistic Regression. It learns one weight per TF-IDF feature and predicts the probability that a review is positive as a sigmoid of the weighted sum. A positive weight pushes a review toward "positive" and a negative weight pushes it toward "negative". Because the output is a probability, the decision threshold can be changed later, which becomes relevant when thinking about how the model would be deployed.

Table 4: Classifier settings

Parameter	Value	Why
class_weight	balanced	Gives the 12% negative class more weight in training so that the model does not simply favour the majority.
max_iter	1000	Enough iterations for the solver to converge.
random_state	42	Reproducible results.
Other settings	defaults	L2 regularisation, C = 1.0. No hyper-parameter search was run.

8.2 The two variants

The **baseline** passes the cleaned text straight to the vectorizer. The **contrast-aware** variant first duplicates the clause after the first contrast marker, as described in Section 5. Everything else (vectorizer, classifier, random seed and train/test split) is identical, which is what makes the comparison fair.

8.3 Training procedure

Each variant is trained once on the 15,854 training reviews and evaluated once on the 3,964 held-out test reviews. No cross-validation or tuning on the test set was done; this keeps the evaluation simple but also means the numbers come from one split (see Section 12). Both models are saved as baseline_model.joblib and contrast_aware_model.joblib.

9. Implementation

The whole project is implemented in Python inside a single Jupyter notebook (pipeline_sentiment_classifier.ipynb). The key libraries are:

- **pandas** and **NumPy** for loading, cleaning and slicing the data;
- **scikit-learn** for TfidfVectorizer, LogisticRegression, train_test_split and the evaluation metrics;
- **matplotlib** for all charts;
- **joblib** for saving the trained models; and the standard `re` module for text cleaning.

The main components are:

- **Loading and labelling:** reads the CSV, drops empty and 3-star reviews, derives the label, the merged category and the length group.
- **Text functions:** clean_text() for the baseline and contrast_aware_text() for the second variant.
- **`SentimentPipeline` class:** one class with a contrast_aware flag. It bundles the vectoriser and classifier and offers fit, predict, predict_proba, evaluate, audit_by_group, top_features and explain methods, plus save. Using one class for both variants guarantees they cannot drift apart in their settings.
- **Audit and explanation code:** groups the test set by category or length, computes negative-review recall for each group, and ranks word weights.

No web application or graphical interface was built; the project is a notebook-based analysis, and the working system is the SentimentPipeline class together with the saved models, which can score any new review text with a single call. The output charts of the notebook (Figures 1 to 9 in this report) serve as the screenshots of the working system. A one-page model card (model_card.pdf) summarising the model, its results and its limits is also included in the repository.

A short snippet shows how the two variants are trained side by side. The complete code is in the repository.

```
for name, contrast_aware in [("baseline", False), ("contrast_aware", True)]:
    model = SentimentPipeline(contrast_aware=contrast_aware)
    model.fit(train_df["review_text"], train_df["sentiment"])
    metrics = model.evaluate(test_df["review_text"], test_df["sentiment"])
```

Listing 1: Training and evaluating both variants with the same pipeline class

An example of the working system is the explanation of one prediction. For the review "I loved the look and the fit was great, but after one wash it shrank terribly", the contrast-aware model predicts *negative* with p(positive) = 0.24. The words that pushed it that way were "shrank" (-0.67), "terribly" (-0.52) and "wash" (-0.45), while "great" (+0.49) pulled the other way. This is shown in Figure 9 (Section 10).

10. Results and Evaluation

Because 88% of the reviews are positive, accuracy alone is a weak guide. I therefore report precision, recall and F1 for the positive class, and, more importantly, **recall for the negative class**, which is the share of genuine complaints that the model catches. ROC AUC and precision-recall curves are shown as threshold-independent views.

10.1 Overall performance

Table 5: Overall test-set performance (n = 3,964)

Metric	Baseline	Contrast-aware
Accuracy	0.915	0.915
Precision (positive)	0.979	0.979
Recall (positive)	0.923	0.923
F1 (positive)	0.950	0.950
Recall (negative)	**0.852** (404 of 474)	**0.857** (406 of 474)
Precision (negative)	0.602	0.601
ROC AUC	0.961	0.961

The two variants are essentially tied. Both are right on 91.5% of reviews, and both catch about 85% of the negative reviews. The one metric that moves is negative recall, from 404 to 406 caught complaints. The 95% Wilson confidence interval for the baseline's negative recall is roughly 82% to 88% , and the contrast-aware interval overlaps it almost completely, so a two-review difference cannot be called a real improvement.

The low precision on the negative class (0.60) is a direct consequence of using balanced class weights on imbalanced data. The model is willing to flag more reviews as negative to find the real complaints, so about four in ten reviews that it flags are actually positive. For triage, where a person reads the flagged reviews, that is an acceptable trade; for automatic action it would not be.

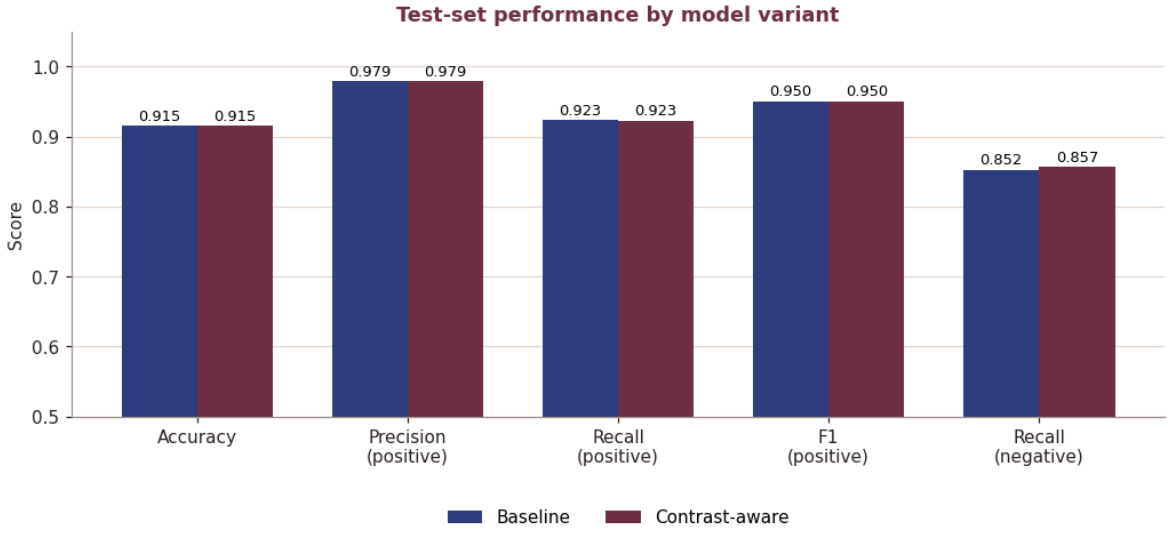


Figure 3: Test-set performance of both variants

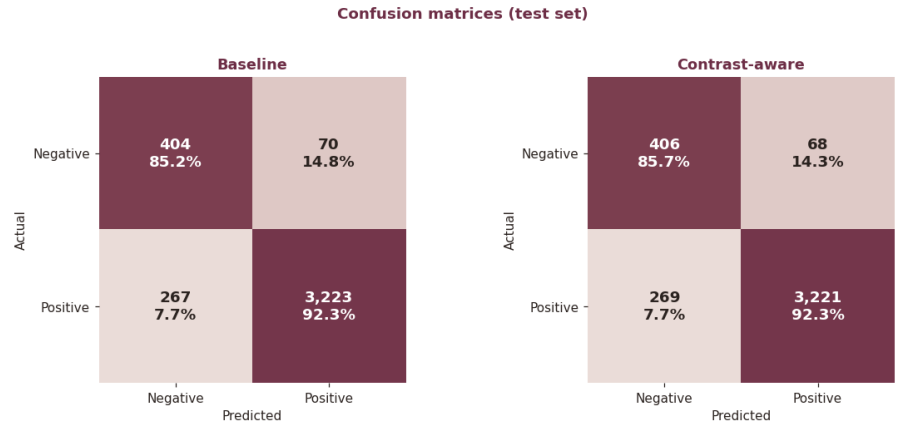


Figure 4: Confusion matrices (counts and row-normalised rates) for both variants

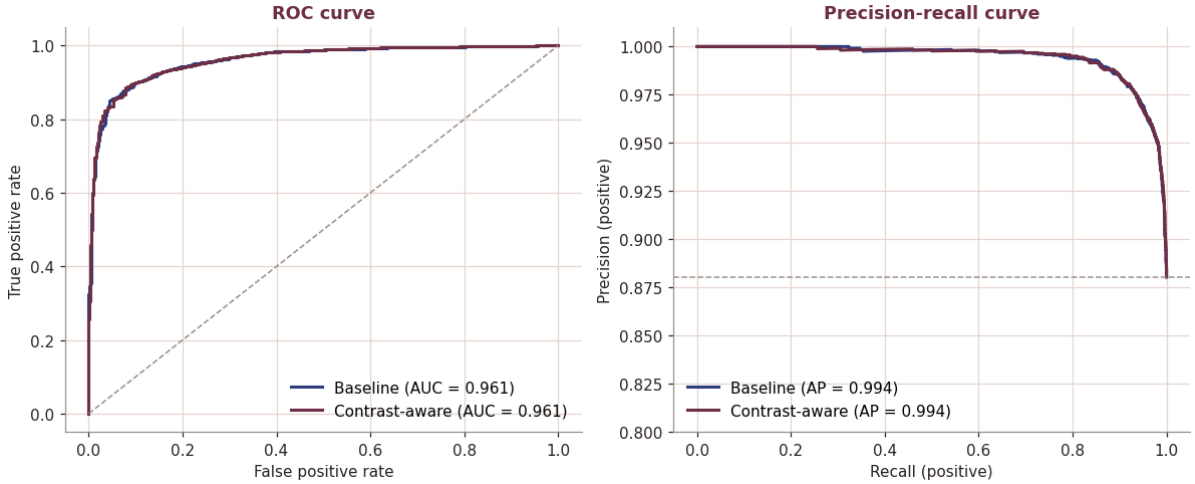


Figure 5: ROC and precision-recall curves for both variants

The confusion matrices make the trade-off visible. Of the 474 negative reviews, the baseline misses 70 (14.8%), while 267 of the 3,490 positive reviews (7.7%) are wrongly flagged as negative. The ROC AUC of 0.961 (average precision 0.994 for the positive class) shows that the underlying ranking of reviews is very good; the errors sit near the decision boundary rather than being spread across the data.

10.2 Audit by product category

This is where the model's average performance starts to hide real differences. Table 6 shows negative-review recall for each category that has at least 100 test reviews.

Table 6: Negative-review recall by product category (test set)

Category	Test reviews	Negative reviews	Caught (baseline)	Recall (baseline)	Recall (contrast)
Sweaters	236	30	29	96.7%	96.7%
Knits	796	99	89	89.9%	86.9%
Dresses	1,109	149	128	85.9%	87.9%
Fine gauge	198	21	18	85.7%	85.7%
Jackets	113	14	12	85.7%	85.7%
Blouses	478	59	50	84.7%	86.4%
Lounge	115	10	8	80.0%	80.0%
Skirts	145	14	11	78.6%	78.6%
Pants	232	22	14	63.6%	68.2%
Jeans	206	13	7	53.8%	53.8%

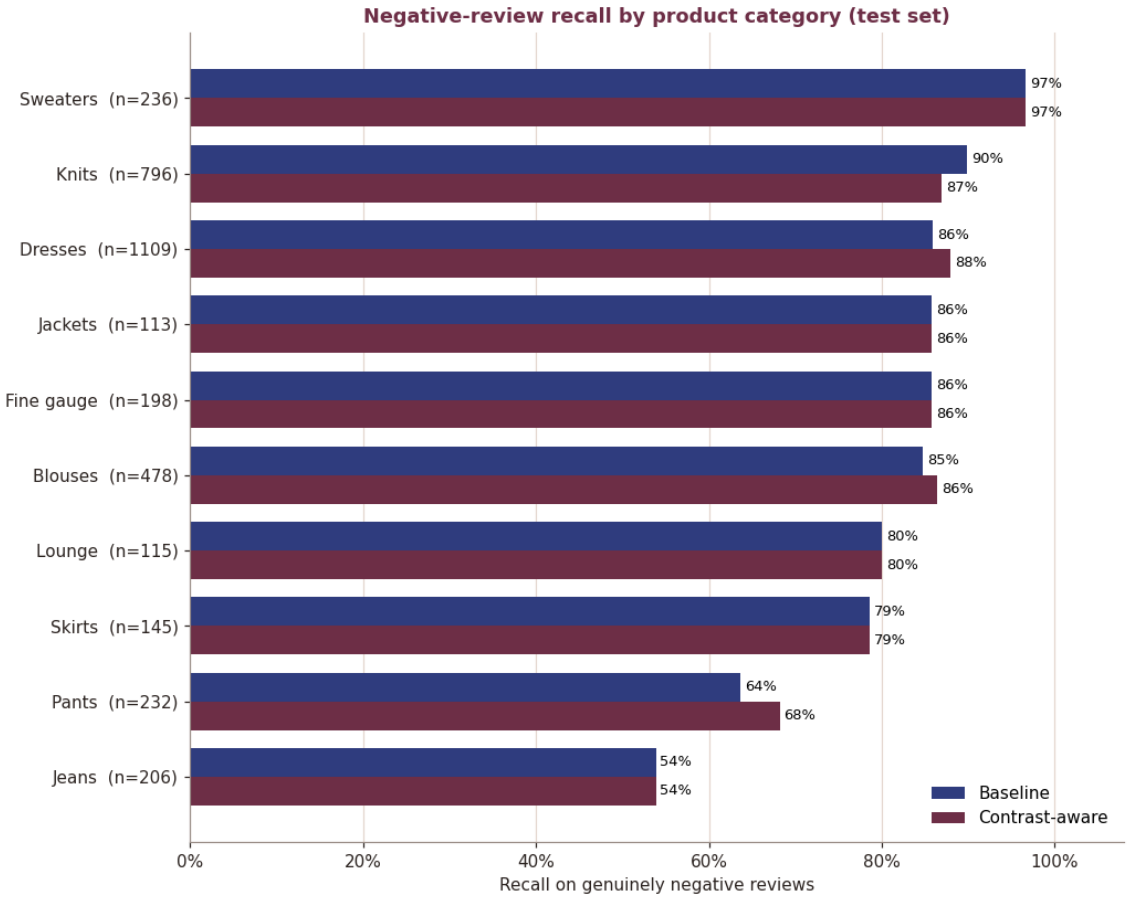


Figure 6: Negative-review recall by product category, both variants (categories with at least 100 test reviews)

Jeans (54%) and Pants (64%) are clearly the weakest, while Sweaters (97%) and Knits (90%) are the strongest, a spread of more than 40 points, even though overall accuracy in each category is around 90%. One caution has to go with this finding: the Jeans figure rests on only 13 negative test reviews (7 caught; 95% interval roughly 29% to 77%) and Pants on 22. The gap is a real warning sign, but the exact percentages are noisy and should be re-tested on more data.

The contrast-aware variant moves the categories in different directions. It helps Pants (+4.6 points), Dresses (+2.0) and Blouses (+1.7), but it lowers Knits by 3.0 points (3 fewer complaints caught). So the contrast idea is not a uniform improvement, and it does nothing for Jeans.

10.3 Audit by review length

Table 7: Performance on short and detailed reviews (cut-off: median of 58 words)

Group	n	Accuracy (base)	Accuracy (contrast)	Neg. recall (base)	Neg. recall (contrast)
Short	2,053	92.4%	92.6%	85.6%	87.3%
Detailed	1,911	90.6%	90.3%	84.9%	84.0%

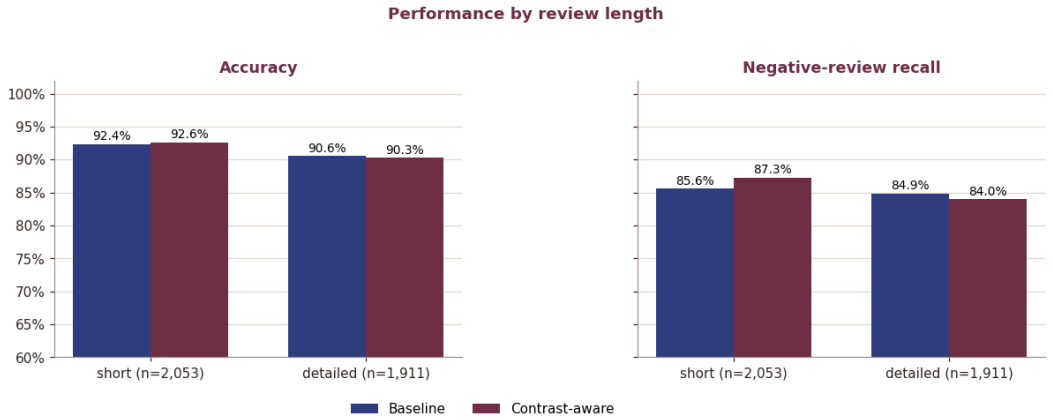


Figure 7: Accuracy and negative-review recall for short and detailed reviews

Short and detailed reviews are handled within 3.3 points of each other on every measure, so review length is not where this model's unevenness comes from. Category is. This is a useful negative result: it saves effort that might otherwise have gone into fixing the wrong problem.

10.4 What the model has learned

Figure 8 shows the ten strongest words for each class. The top positive features are *love*, *perfect*, *little*, *comfortable* and *great*. The top negative features are *disappointed*, *cheap*, *returned*, *unflattering* and *huge*. These are real sentiment words rather than brand names or category labels, which is reassuring. Two things stand out, though. "Returned", "return", "returning" and "huge" are strong negative signals, which suggests the model has partly learned *sizing and return language* instead of pure opinion. And the bigram "wanted love" (from "wanted to love") is among the strongest negatives, a nice example of why bigrams were worth including.

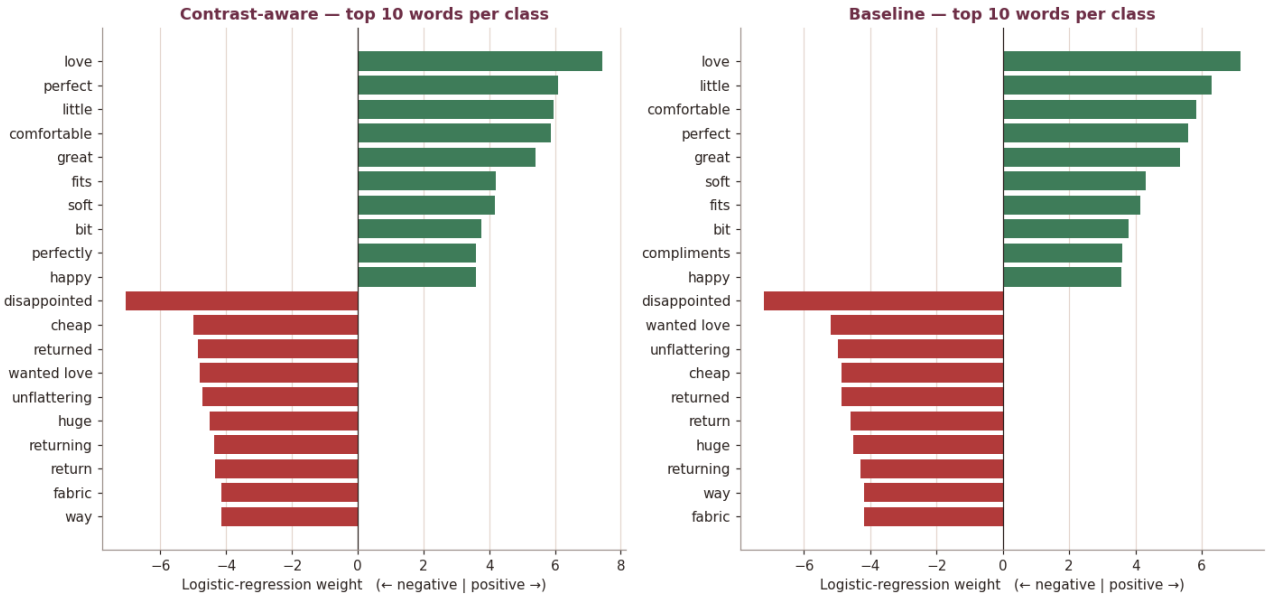


Figure 8: Ten strongest words per class for each variant (green = pushes toward positive, red = toward negative)

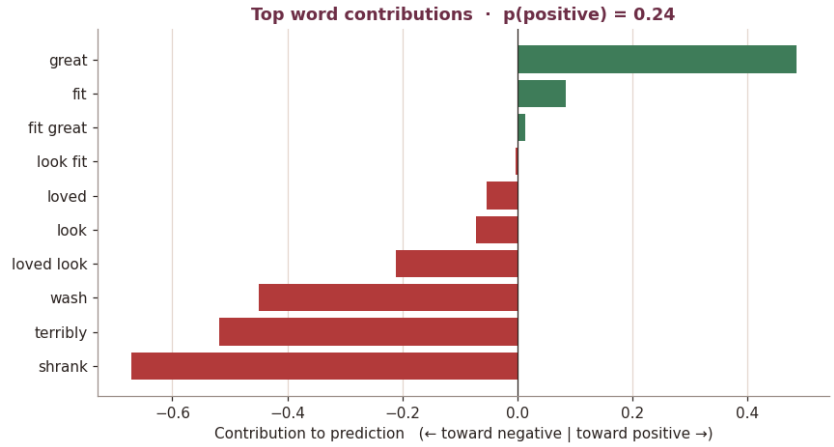


Figure 9: Per-word contributions for a mixed review: "I loved the look and the fit was great, but after one wash it shrank terribly"

Figure 9 shows the contrast-aware model reading a mixed-signal review correctly. Despite the cheerful opening, the words after "but" dominate and the review is classified as negative (p(positive) = 0.24). This is the behaviour the contrast-aware transform was designed to produce, although, as the aggregate numbers show, it fires on far fewer reviews than hoped.

11. Error Analysis / NLP Failure Analysis

To understand the numbers, I re-ran the baseline on the same split and read the misclassified reviews. The model makes 337 errors on the test set: 70 genuine complaints that were missed (false negatives for the negative class) and 267 positive reviews that were wrongly flagged as negative. The errors cluster into a few recognisable patterns.

11.1 Missed complaints (70 reviews)

Many missed complaints do not contain any obviously negative word. The customer is describing a problem with fit, length or material in a calm tone, and a model that learned "negative = angry vocabulary" cannot see it.

Table 8: Examples of negative reviews that the baseline classified as positive

Review (as written)	Stars / p(pos)	Likely reason for failure
"Totally see through."	1 / 0.60	Very short; "see through" is a complaint but is not a learned negative phrase.
"Pretty fabric, nice style, took it back."	2 / 0.65	Positive adjectives outweigh the one clue ("took it back"). The verdict is implicit.
"This sweater is very long, could be worn as a dress at 5'3 !"	2 / 0.73	A fit complaint expressed neutrally; no sentiment words. Digits are also stripped.
"Nice and colorful dress but runs very oversized. material mutes the colors not as vibrant in person."	1 / 0.80	Praise comes first and the negatives are mild, so positive words win. "Not" is removed as a stop-word.
"Fits large, wrinkles easily and not what i expected for the price."	2 / 0.62	Negation ("not what i expected") is invisible once "not" is dropped.
"C'mon. the accent is wrong! a simple google search would prevent this."	2 / 0.53	Domain-specific complaint about a foreign-language word; relies on world knowledge.

Of the 70 missed complaints, 54 were 2-star and 16 were 1-star reviews, so most are the milder complaints. The average predicted probability of "positive" for these reviews is 0.64, and 18 of them scored above 0.70, which means the model was not on the fence: it was fairly sure and wrong. About 73% of them contain one of the contrast words, which confirms how common the "praise, then problem" structure is, and also why the contrast-aware idea was worth trying.

11.2 False alarms (267 reviews)

The false alarms are, if anything, more interesting, because on reading them many are not clearly the model's fault.

Table 9: Examples of positive reviews that the baseline classified as negative

Review (as written)	Stars / p(pos)	Likely reason for failure
"I loved the shirt... unfortunately it didn't look good on me. but i would recommend this shirt to anyone who can pull off the look"	5 / 0.05	Mixed opinion; "unfortunately" and "didn't" dominate, but the reviewer actually recommends the product.
"Very cute shorts. sadly, after one wash they became a lint magnet so had to be returned. disappointing quality."	5 / 0.02	The text is a genuine complaint; the 5-star rating looks like a label error.
"Runs huge! returning the m for a s; ... very soft, wonderful colour."	4 / 0.34	Fit problem plus return language, but the reviewer likes the product and just needs another size.
"I love this top. the only thing negative is how thin and cheap the material feels."	4 / 0.37	"Cheap" is a heavy negative word that overrides "love"; the reviewer weighs it as a minor issue.
"For reference i am wearing a medium in the photos and my measurements are 38-30-40."	4 / 0.38	No opinion at all; a sizing note that happens to sit in a 4-star review.
"As cute in person as the picture shows. good quality, worth the money!"	5 / 0.18	Clearly positive; probably hurt by the fact that "as", "in person" and "picture" are common in complaints ("not as pictured").

Several patterns are visible. First, 165 of the 267 false alarms are 4-star reviews, and only 102 are 5-star, so the model struggles most on the milder positives. Second, about 24% of false alarms mention returning the item, compared with 6% of all test reviews, and 69% mention a sizing word (large, small, tight and so on). These are customers who liked the product but had to send it back because of size. The model has learned that "return" means trouble, which is usually true, but not always. Third, in a few cases the text is an outright complaint attached to a high rating, such as the lint-magnet shorts. In those cases the *label* is wrong, not the model.

11.3 Underlying causes

- **Negation and contrast words are discarded.** The stop-word list removes *not*, *no*, *never*, *but*, *however* and *very*, so the model sees "disappointing quality" and "not what i expected" with the negation gone. Apostrophes are also stripped, so "didn't" becomes "didn t" and the "t" is dropped as a one-letter token.
- **A bag of words has no sentence structure.** It cannot tell "loved it but too small" from "too small but loved it". The contrast trick only approximates that with a crude duplication rule keyed to the first contrast word.

- **Star ratings measure the product and the fit together.** A customer who loves a dress but ordered the wrong size may give four stars and write a text that reads as a complaint. The model is penalised for being faithful to the words.
- **Vocabulary gaps in small categories.** Jeans and Pants reviews are dominated by fit vocabulary ("waist", "rise", "stretch") that is less predictive of sentiment, and the training set contains few negative reviews for those categories. I have not tested this explanation directly, so it remains a hypothesis.
- **No handling of sarcasm.** None of the sampled errors was clearly sarcastic, but a model built on word counts would have no way of detecting it if it were.

12. Ethical Considerations and Limitations

12.1 Fairness across categories

The most important ethical point in this project is also the main technical finding: the model does not serve all product categories equally. If it were used to prioritise customer complaints, sellers of Jeans and Pants would have more of their unhappy customers go unnoticed than sellers of Sweaters. In a marketplace, that can translate into unequal treatment of sellers or products, even though nobody intended it. This is why the recommended use is as a first-pass filter with a human in the loop, with performance monitored per category and not only overall.

12.2 Limitations

6. **Uneven category performance.** Negative recall is 54% for Jeans and 64% for Pants against 97% for Sweaters. Overall accuracy of 91.5% hides this.
7. **Small samples.** The weakest categories rest on 13 and 22 negative test reviews. The same caution applies to the 2-review gain of the contrast-aware model, which is within noise.
8. **Many false alarms.** Negative precision is about 0.60, so roughly four in ten flagged reviews are actually positive.
9. **Word-counting model.** TF-IDF cannot handle negation, sarcasm or sentence structure, and the contrast-aware transform is only an approximation.
10. **Proxy labels.** Sentiment is derived from star ratings, which, as Section 11 shows, do not always match the text. There is no neutral class, since 3-star reviews were removed.
11. **Narrow scope.** The data covers one English-language, women's-clothing retailer. The model should not be assumed to work on other products, other languages or other writing styles.
12. **Evaluation design.** A single train/test split was used, without cross-validation, and no hyper-parameters were tuned.

12.3 Privacy and demographics

The dataset is public and anonymised; no names or contact details are present in the fields used. The file does contain reviewer age (18 to 99, median 41), but age was not used as a feature and I did not check whether the model performs differently for different age groups or writing styles. That is a gap, and a deployed system would need that check.

12.4 Recommended use

The model is suitable for first-pass triage that surfaces likely complaints for a person to read, and for aggregate sentiment analytics over large review collections. It should **not** be used for automatic actions such as removing reviews, penalising sellers or issuing refunds, and it should not be used on Jeans or Pants without review, or on neutral sentiment, non-English text or products outside women's clothing.

13. Conclusion

This project built a positive/negative classifier for women's clothing reviews using TF-IDF features and Logistic Regression, compared a baseline with a contrast-aware variant, and then audited the result. Both variants reach 91.5% accuracy, an F1 of 0.95 on the positive class and a ROC AUC of 0.961, and catch about 85% of the negative reviews.

Looking at the objectives one by one: a working, saved and reproducible classifier was built; the two variants were compared fairly on the same split; the audit showed that product category, and not review length, is where the model is unreliable; the word weights and a per-review breakdown made the predictions explainable; and the limits and intended use were written down. The one

hypothesis that did not deliver much was the contrast-aware fix. It caught 2 more of 474 complaints, helped Pants, Dresses and Blouses, hurt Knits, and did nothing for Jeans, so it is best described as a small and uneven gain.

The main thing I take away is how little a good accuracy score says on its own. On this dataset the first number I saw was reassuring, and it was only by splitting the results by category and by reading the actual mistakes that the real picture appeared: a model that is excellent at some things and quietly poor at others, and a set of labels that are not always as clean as they look. I also learned, rather late, to check what the library defaults do to my text. Finding that the stop-word list had been removing "not" and "but" all along was a useful reminder that default settings are decisions too.

14. Future Scope

- **Clause-Level Parsing.** Clause-level parsing breaks a complex sentence into smaller clauses and identifies their relationship.
- **Negation-Scope Detection.** Negation-scope detection identifies a negation word and determines exactly which part of the sentence it affects.
- **Try a contextual model.** Fine-tune a transformer such as BERT , which reads whole sentences and should cope better with mixed opinions, and compare it against this linear baseline, both on overall metrics and on the category audit.
- **Fix the weak categories.** Collect or oversample negative Jeans and Pants reviews, and test whether category-specific thresholds or models close the gap.
- **Tune the threshold toward recall** for the weak categories and measure the cost in precision, since the model already outputs probabilities.
- **Stronger evaluation.** Use cross-validation and confidence intervals for every audit slice, and audit by reviewer age group and writing style.
- **Address label noise.** Add a neutral class or use the review text together with the "Recommended" flag to reduce mismatches between the star rating and the text.
- **Build a small review-triage app** that shows the prediction, the probability and the contributing words, so that a human reviewer can see why a review was flagged.

15. References

References are given in APA style.

Brooks, N. (2018). *Women's E-Commerce Clothing Reviews* [Data set]. Kaggle. https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews

scikit-learn developers. (2026). *scikit-learn documentation: TfidfVectorizer, LogisticRegression, ENGLISH_STOP_WORDS*. https://scikit-learn.org/stable/

16. GitHub Repository

The code, data file, notebook and model card are available in the project repository.

- **GitHub Repository Link:** https://github.com/Tanay006/product_review_sentiment_classification

The repository contains pipeline_sentiment_classifier.ipynb (the complete implementation), Reviews.csv (the dataset), model_card.pdf (one-page model card) and Readme.md (a short write-up of the project). Running the notebook from top to bottom with random_state = 42 reproduces every number and chart in this report.

17. Annexure

A. Sample input and output

Table 10: Sample predictions

Input review	Contrast-aware prediction	p(positive)
I loved the look and the fit was great, but after one wash it shrank terribly.	Negative	0.24

B. Top words by weight (contrast-aware model)

Table 11: Ten strongest words per class

Positive word	Weight	Negative word	Weight
love	+7.44	disappointed	-7.04
perfect	+6.10	cheap	-5.01
little	+5.96	returned	-4.86
comfortable	+5.87	wanted love	-4.82
great	+5.42	unflattering	-4.72
fits	+4.21	huge	-4.51
soft	+4.16	returning	-4.37
bit	+3.75	return	-4.34
perfectly	+3.59	fabric	-4.15
happy	+3.58	way	-4.14

C. Class report (baseline, test set)

Table 12: Per-class precision, recall and F1 (baseline)

Class	Precision	Recall	F1-score	Support
Negative	0.60	0.85	0.71	474
Positive	0.98	0.92	0.95	3,490
Accuracy			0.91	3,964
Macro average	0.79	0.89	0.83	3,964
Weighted average	0.93	0.91	0.92	3,964